CI/CD & Deployment

How code moves from developer laptop through GitHub Actions and Workload Identity Federation into me-central2 Cloud Run — repeatably, from any authorized machine, with no dependency on any specific developer workstation.

PREPARED FOR
PhysioGuidance Client Team

DATE
2026-08-17

VERIFIED AGAINST
Deployed commit `d396b040`

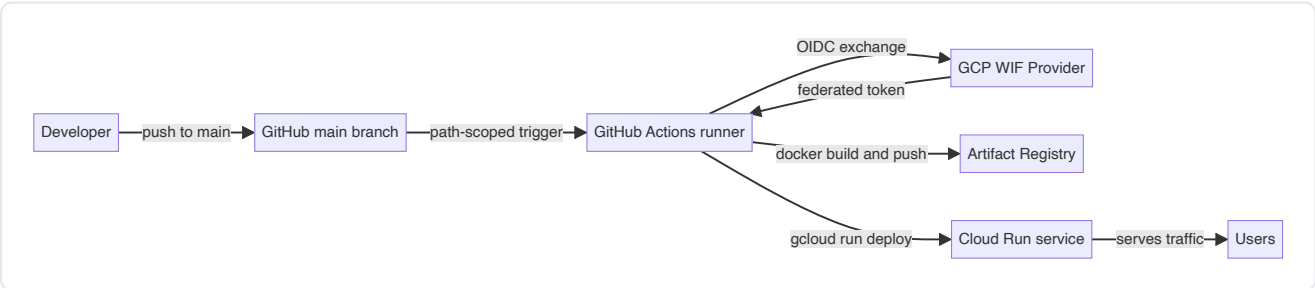
PHYSIOGUIDANCE
Riyadh · Saudi Arabia
me-central2 · GCP Cloud Run

Table of contents

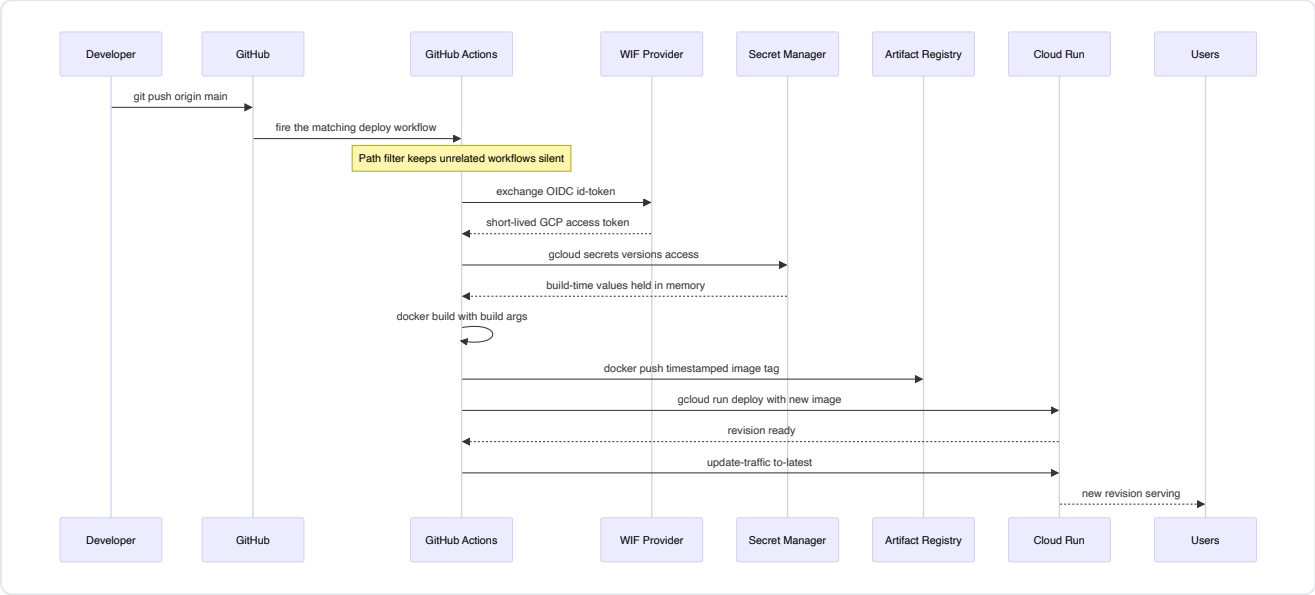
1. Executive summary — how code becomes running production
2. Deployment pipeline diagram
3. GitHub Actions workflows
4. Path-based triggering — who deploys what
5. Workload Identity Federation (WIF) — auth without long-lived keys
6. Per-workflow required secrets
7. Deploy scripts (`infra/*/deploy.sh`)
8. Rollback procedure
9. Manual deployment (fallback when CI is down)
10. Clean-machine deployment demonstration
11. Independence — no dependency on developer machine
12. Component status matrix

1. Executive summary

Every merge to `main` triggers a **path-scoped** GitHub Actions workflow that builds a Docker image on the runner, pushes it to GCP Artifact Registry, and rolls a new revision on Cloud Run. The workflow authenticates to GCP via **Workload Identity Federation** (WIF) — no long-lived service-account keys are stored in GitHub. The same `infra/<service>/deploy.sh` scripts CI invokes also work from a clean developer machine, so any authorized operator with `gcloud` , `docker` , and repo access can deploy without depending on a specific workstation.



2. Deployment pipeline diagram



Every image is tagged twice: once with a timestamp (`YYYYMMDD-HHMMSS` , used as the immutable rollback anchor) and once with `:latest` . Cloud Run pins the deployed revision to the timestamped tag, so future rollbacks can target the exact historical image.

3. GitHub Actions workflows

Located in `.github/workflows/` . Each workflow deploys exactly one Cloud Run service.

Workflow file	Deploys	Trigger
`deploy-backend.yml`	`physioguidance-api`	Push to `main` touching `backends/**` , `infra/backend/**` , `infra/config.sh` , or the workflow file, plus `workflow_dispatch`
`deploy-frontend.yml`	`physioguidance-web`	Push to `main` touching `frontend/**` , `infra/frontend/**` , `infra/config.sh` , or the workflow file, plus `workflow_dispatch`
`deploy-worker.yml`	`physioguidance-api-worker`	`workflow_run` chained after a successful **Deploy Backend** run, push touching `infra/worker/**` or the workflow file, plus `workflow_dispatch`
`deploy-strapi.yml`	`physioguidance-cms`	Push to `main` touching `strapi/**` , `infra/strapi/**` , `infra/config.sh` , or the workflow file, plus `workflow_dispatch`

Handled manually by design:

- `physioguidance-malware-scanner` — deployed via `infra/malware-scanner/deploy.sh`. Manual so a routine app deploy never recycles the scanner that guards all three user-upload buckets, and so a re-run refreshes the ClamAV signatures baked in at image build.

3.1 Common workflow shape

Every deploy workflow follows the same template:

```yaml
on:
  push:
    branches: [main]
    paths:
      - '<service>/**'
      - 'infra/<service>/**'
      - 'infra/config.sh'
      - '.github/workflows/deploy-<service>.yml'
  workflow_dispatch:

concurrency:
  group: deploy-<service>-${{ github.ref }}
  cancel-in-progress: false   # a running deploy is never interrupted

jobs:
  deploy:
    runs-on: ubuntu-latest
    timeout-minutes: 30
    permissions:
      contents: read
      id-token: write   # required for WIF
    steps:
      - uses: actions/checkout@v4
      - uses: google-github-actions/auth@v2      # WIF
      - uses: google-github-actions/setup-gcloud@v2
      - run: gcloud auth configure-docker ${{ secrets.GCP_REGION }}-docker.pkg.dev --quiet
      - run: bash infra/<service>/deploy.sh --skip-pull
```

The `--skip-pull` flag tells the deploy script to trust the code already checked out by `actions/checkout` and skip the `git pull` step it otherwise runs on a developer machine.

3.2 Worker chained deploy

`deploy-worker.yml` is the only workflow that reacts to another workflow finishing. It listens for **Deploy Backend** to complete on `main`, runs only when the backend run's `conclusion == 'success'`, and reuses the exact image the backend deploy just pushed (the worker container runs the same monolithic Nest app, just with different runtime flags). The path filter deliberately excludes `backends/**` so the worker workflow never races the backend workflow to build the same image twice.



4. Path-based triggering — who deploys what

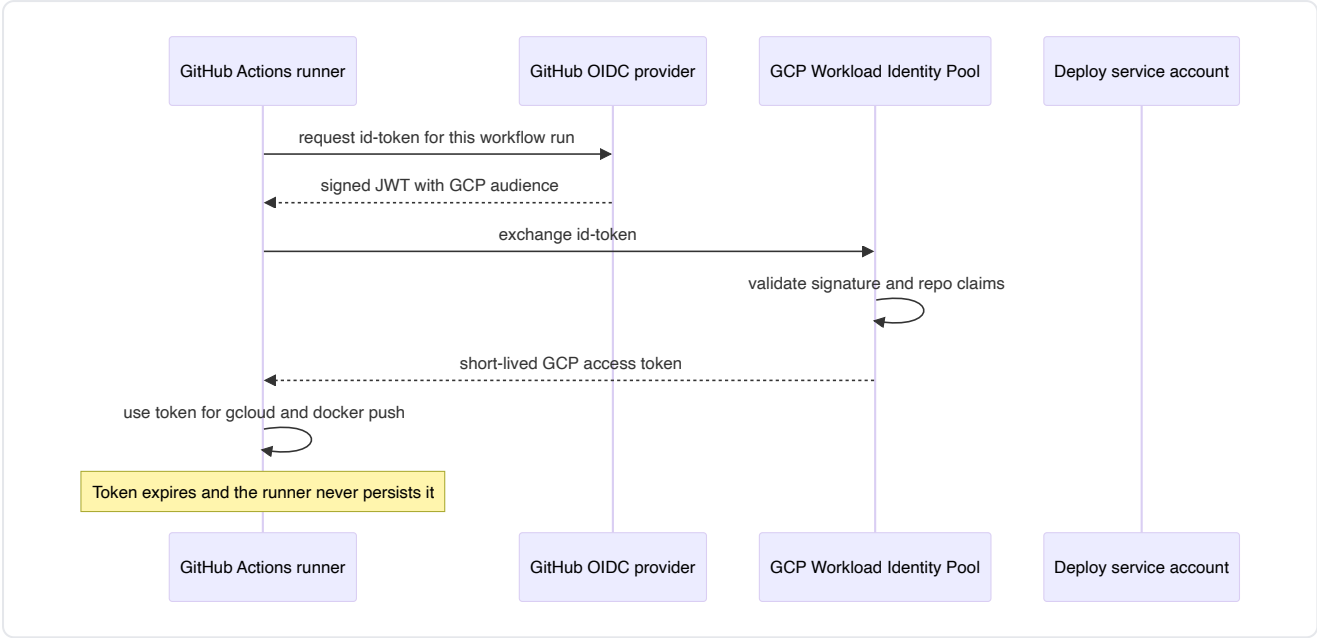
Which workflows fire on a given commit depends on which paths it touches. Use this table to predict deploy fan-out before pushing.

Files touched	Backend	Frontend	Worker	Strapi
`backends/**`	Yes		Yes (chained)	
`frontend/**`		Yes		
`strapi/**`				Yes
`infra/backend/**`	Yes		Yes (chained)	
`infra/frontend/**`		Yes		
`infra/worker/**`			Yes	
`infra/strapi/**`				Yes
`infra/config.sh`	Yes	Yes	Yes (chained)	Yes
`.github/workflows/deploy-<x>.yml`	Only that one	Only that one	Only that one	Only that one
Docs only (`*.md` , `PhysioGuidance-Handover/**`)				
Repo root files (README etc.)				

Doc-only commits deliberately trigger no deploy, keeping CI minutes and Cloud Run revision history focused on real code changes.

5. Workload Identity Federation (WIF)

GitHub Actions authenticates to GCP via OpenID Connect. GCP trusts GitHub's OIDC provider for this repo, so no service-account key material is stored in GitHub.



5.1 Configured provider

- **Pool:** `github-actions-pool`
- **Provider:** `github-actions-provider`
- **Deploy SA:** `github-actions-deployer@nelc-elearning-platform-479319.iam.gserviceaccount.com`

5.2 Deploy SA roles

The deploy SA holds only the roles needed to build and roll a Cloud Run revision:

Role	Purpose
`roles/run.developer`	Deploy Cloud Run revisions
`roles/artifactregistry.writer`	Push images to Artifact Registry
`roles/iam.serviceAccountUser`	Attach per-service runtime SAs on deploy
`roles/secretmanager.secretAccessor` (per secret)	Read build-time secrets: `NEXT_PUBLIC_RECAPTCHA_SITE_KEY`, `NEXT_PUBLIC_SENTRY_DSN`, `SENTRY_AUTH_TOKEN`

Full WIF setup detail is in wif-setup-for-amna.md, including the exact `gcloud iam workload-identity-pools` commands to recreate the pool.

---

6. Per-workflow required secrets

Set in GitHub → Repo settings → Secrets and variables → Actions. These are **GitHub Actions secrets**, distinct from GCP Secret Manager entries.

Secret	Used by	Purpose
`WIF_PROVIDER`	All workflows	Full workload-identity provider resource name
`WIF_SERVICE_ACCOUNT`	All workflows	Deploy SA email (`github-actions-deployer@`)
`GCP_PROJECT_ID`	All workflows	`nelc-elearning-platform-479319`
`GCP_REGION`	All workflows	`me-central2`

No GCP JSON keys are stored in GitHub Actions secrets — WIF eliminates that surface. Runtime application secrets (DATABASE_URL, JWT_SECRET, gateway keys, etc.) live only in **GCP Secret Manager** and are mounted into Cloud Run via `--set-secrets` .

---

7. Deploy scripts (`infra/*/deploy.sh`)

Every workflow ends by calling `bash infra/<service>/deploy.sh --skip-pull` . Keeping the deploy logic in shell files means the same script works identically in CI and on a developer laptop.

7.1 Backend deploy script (`infra/backend/deploy.sh`)

1. **Source** `infra/config.sh` for shared env vars: project id, region, image registry, service names, sizing.
2. **Compute a timestamped image tag** (`${DOCKER_REGISTRY}/physioguidance-api:YYYYMMDD-HHMMSS`) plus a `:latest` alias.
3. `docker build --platform linux/amd64` from `backends/` using `infra/backend/production/Dockerfile` .
4. `docker push` both tags to Artifact Registry.
5. **Resolve URLs** for CORS (frontend custom domain + both Cloud Run URL formats) and payment callbacks.
6. **Assemble env vars and secrets**: storage buckets, Cloud Tasks dispatch config, Redis, Zoom, Moyasar / Tamara / Tabby gateways, NELC LRS endpoints, Sentry. Existing values for `MOYASAR_MODE` , `SKIP_BULLMQ_WORKERS` , `SKIP_CRONS` , `EXTERNALIZED_CRONS` are read back from the current revision and preserved so a routine deploy never silently flips a mode.
7. `gcloud run deploy physioguidance-api` with:

- `--service-account=api-runtime@…`
- `--port=8080 --memory=1Gi --cpu=1 --cpu-throttling`
- `--min-instances=0 --max-instances=10`
- `--add-cloudsql-instances=nelc-elearning-platform-479319:me-central2:physioguidance-db-me`
- `--vpc-connector=physio-vpc-connector-me --vpc-egress=all-traffic`
- `--set-env-vars` and `--set-secrets` built above

8. **`gcloud run services update-traffic --to-latest`** so the new revision serves immediately, even if traffic had previously been pinned to a specific revision.

7.2 All five deploy scripts

Script	Deploys	Runtime SA	Notable flags
`infra/backend/deploy.sh`	`physioguidance-api`	`api-runtime@`	1Gi / 1 vCPU, max=10, `--cpu-throttling`, VPC connector, Cloud SQL socket, public
`infra/frontend/deploy.sh`	`physioguidance-web`	`web-runtime@`	512Mi / 1 vCPU, max=3, `--allow-unauthenticated`, `NEXT_PUBLIC_*` build args
`infra/worker/deploy.sh`	`physioguidance-api-worker`	`worker-runtime@`	Reuses backend image, 1.5Gi / 1 vCPU, min=max=1, `--no-cpu-throttling`, `--no-allow-unauthenticated`, `SKIP_CRONS=true`
`infra/strapi/deploy.sh`	`physioguidance-cms`	`strapi-runtime@`	512Mi / 1 vCPU, max=2, Cloud SQL socket, GCS upload provider (`physioguidance-cms` bucket)
`infra/malware-scanner/deploy.sh`	`physioguidance-malware-scanner`	`malware-runtime@`	Also wires Eventarc triggers on the three user-upload buckets; fresh build refreshes ClamAV signatures

Additionally, `infra/scripts/migrate-production.sh` runs Drizzle migrations against Cloud SQL through `cloud-sql-proxy` on port 5433. Run it manually before any deploy that changes DB schema — see developer-guide.md for the full migration workflow.

7.3 Worker deploy script (`infra/worker/deploy.sh`)

The worker deploy is intentionally distinct:

- It **reuses** the `physioguidance-api:latest` image built by the backend script — never rebuilds.
- It sets `--no-cpu-throttling` so BullMQ consumers can poll Redis continuously.
- It sets `SKIP_CRONS=true` so the in-container `@Cron` scheduler stays off (crons run on `physioguidance-api`, which itself sets `SKIP_BULLMQ_WORKERS=true` — the two flags together prevent double execution).
- It sets `min=max=1` — a single writer against the shared BullMQ queue, with per-queue concurrency handling parallelism.
- It sets `--no-allow-unauthenticated` — the worker exposes no public URL; all work arrives via Redis or the shared Postgres DB.

---

8. Rollback procedure

Cloud Run keeps every past revision. Rollback is one click or one command.

8.1 Console

1. GCP Console → Cloud Run → click the service.
2. **Revisions** tab → find the previous known-good revision (image tag = the timestamp of the known-good deploy).
3. Click the row menu → **Manage traffic** → route 100% to the previous revision → **Save**.

8.2 CLI

```
# List current traffic split
gcloud run services describe physioguidance-api --region=me-central2 \
  --format="value(status.traffic[].revisionName,status.traffic[].percent)"

# Roll back to a specific revision
gcloud run services update-traffic physioguidance-api --region=me-central2 \
  --to-revisions=physioguidance-api-00130-abc=100
```

Traffic switches within seconds; in-flight requests continue on the current revision until they complete.

When to roll back:

- Sentry error rate spikes well above baseline right after a deploy — roll back and investigate.
- Deploy reports healthy but users report broken pages — roll back, then look at frontend cache or route boundaries.
- A deploy that also ran a DB migration cannot be rolled back by traffic-shift alone — restoring the DB from the PITR window to a point before the migration is required first. See regional-failure-playbook.md.

---

9. Manual deployment (fallback when CI is down)

If GitHub Actions is unavailable during a GitHub incident:

```
# From any machine with gcloud auth and the repo checked out:
cd /path/to/physioguidance
git checkout main && git pull
gcloud auth login        # if needed
gcloud config set project nelc-elearning-platform-479319

# Deploy affected service(s) — same scripts CI runs:
./infra/backend/deploy.sh
./infra/frontend/deploy.sh
./infra/worker/deploy.sh
./infra/strapi/deploy.sh
./infra/malware-scanner/deploy.sh
```

The scripts perform the same build → push → deploy → traffic-shift steps they run in CI. Never run `docker compose` or `docker run` against production data — see platform-owner-guide.md §10.

---

10. Clean-machine deployment demonstration

Amna's Item 2 explicitly requires demonstrating that another authorized developer can deploy from a clean machine.

Prerequisites (one-time per operator)

- macOS or Linux
- `gcloud` CLI
- Docker Desktop or equivalent
- Git
- IAM on the GCP project: `roles/run.developer`, `roles/artifactregistry.writer`, `roles/iam.serviceAccountUser`, plus `secretmanager.secretAccessor` on the specific build-time secrets

Steps

```
# 1. Clone the repo
git clone https://github.com/physioguidance1-arch/physioguidance.git
cd physioguidance

# 2. Authenticate to GCP
gcloud auth login
gcloud auth configure-docker me-central2-docker.pkg.dev --quiet
gcloud config set project nelc-elearning-platform-479319

# 3. Deploy any service
./infra/backend/deploy.sh
```

All runtime secrets are pulled from GCP Secret Manager at container start. Build-time secrets (`NEXT_PUBLIC_*` values, `SENTRY_AUTH_TOKEN`) are read via `gcloud secrets versions access` inside the deploy script. No `.env` file is ever required on the operator's machine.

---

11. Independence — no dependency on developer machine

Confirmed:

- All deployment scripts live in the repo (`infra/`).
- All CI workflows live in the repo (`.github/workflows/`).
- All runtime secrets live in **GCP Secret Manager** — none only on any developer's machine.
- All build-time secrets are read via `gcloud secrets versions access` — never from local `.env` files.
- WIF removes the need for any JSON key file anywhere.
- Custom domains and DNS are configured at the domain registrar and Cloud Run URL mapping, not on any developer machine.
- Handover documentation (gcp-deployment.md, wif-setup-for-amna.md, developer-guide.md, this document) is in the repo.

---

12. Component status matrix

Using Amna's four-category vocabulary.

Component	Status	Notes
GitHub Actions workflows for backend, frontend, worker, Strapi	Implemented	`.github/workflows/deploy-{backend,frontend,worker,strapi}.yml`
Workload Identity Federation (OIDC) auth to GCP	Implemented	No long-lived keys in GitHub
Per-service runtime SAs applied on every deploy	Implemented	`api-runtime@`, `web-runtime@`, `worker-runtime@`, `strapi-runtime@`, `malware-runtime@`
Path-scoped triggering (docs-only commits skip CI)	Implemented	Path filters in each workflow
Chained worker deploy after successful backend	Implemented	`workflow_run: [Deploy Backend]` in `deploy-worker.yml`
Timestamped image tags + `:latest` on every deploy	Implemented	Timestamp is the rollback anchor
Traffic pinned to latest revision on every deploy	Implemented	`gcloud run services update-traffic --to-latest`
Rollback via Cloud Run revision traffic-shift	Implemented	Documented in §8
Clean-machine deployment	Implemented	Reproducible per §10
Independence from any developer workstation	Implemented	Confirmed §11
Malware scanner deploy	Implemented	Handled manually via `infra/malware-scanner/deploy.sh` by design

13. Where to look next

- **What each deploy script configures on Cloud Run** → gcp-deployment.md §2, §9
- **The WIF pool and provider setup detail** → wif-setup-for-amna.md
- **Local development, migrations, testing** → developer-guide.md
- **Rollback and incident response** → platform-owner-guide.md §9